



■ CS177 Bioinformatics: Software Development and Applications

Lecture 6: Introduction to Deep Learning

Part 1. ANN and CNN

Jie Zheng (郑杰)

PhD, Associate Professor

School of Information Science and Technology (SIST), ShanghaiTech University

Spring, 2024



Learning objectives



- After this lecture, we will be able to:
 - Explain basics of artificial neural networks
 - Describe basic ideas, techniques and challenges of deep learning
 - Tell the biological background and rationale behind convolutional neural networks (CNNs)
 - Describe main components of a CNN
 - Explain how convolutional layers and pooling layers work
 - List a few famous example CNN architectures





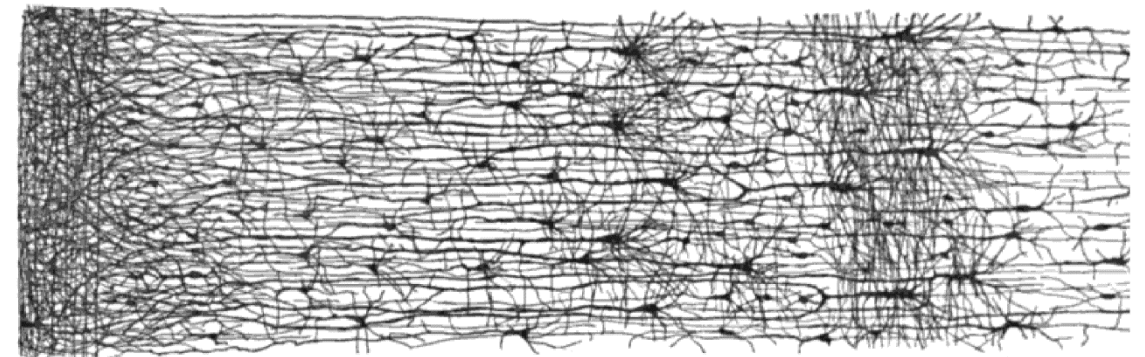
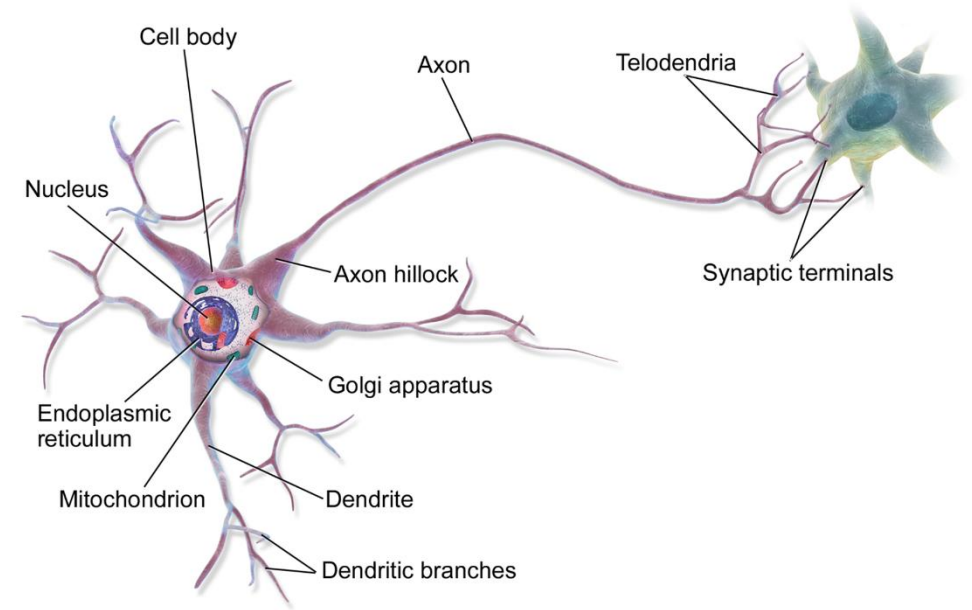
Artificial Neural Network

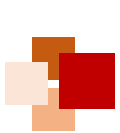


Biological neurons



- Components in a biological neuron
 - Cell body
 - Dendrite
 - Axon
 - Synapses
- Communications among neurons
 - Neurons receive signals (i.e. short electrical impulses) from other neurons via synapses
 - When a neuron receives an enough number of signals, it fires its own signals
- Biological neural networks
 - Neurons are organized in consecutive layers

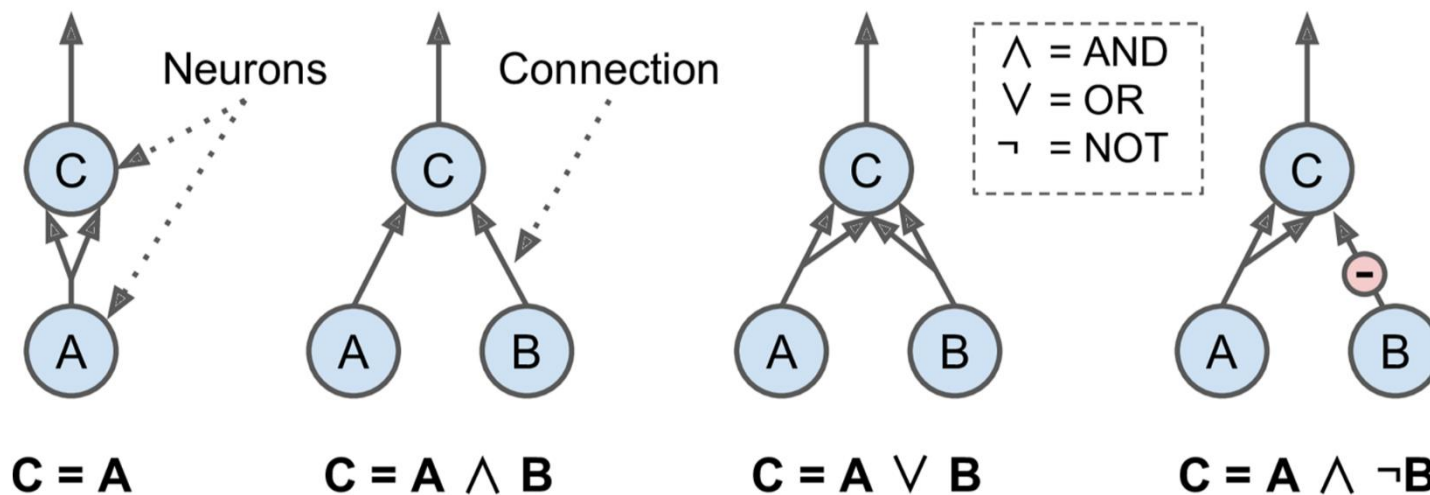




Logical computations with neurons

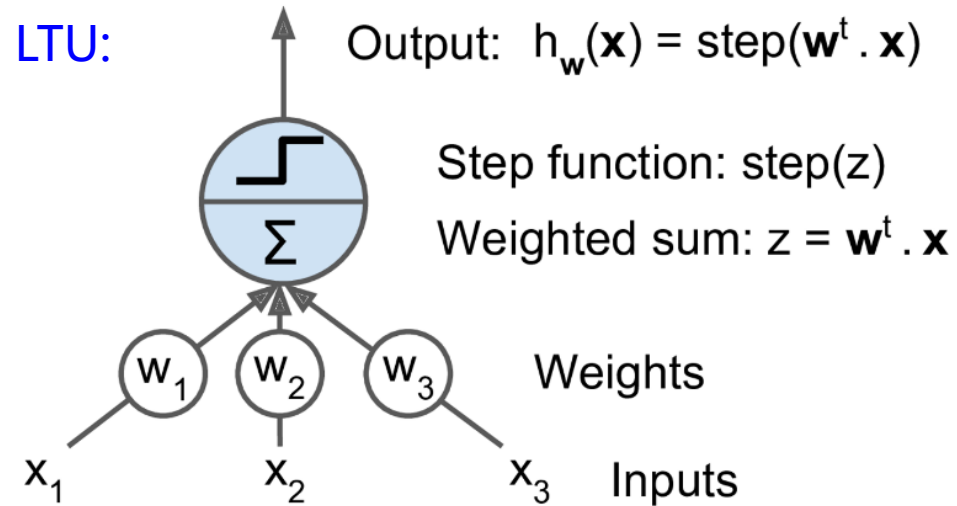


- **Artificial neural network** (McCulloch and Pitts, 1943): A simplified computational model of how biological neurons might work together in animal brains to perform computations using **propositional logic**
- An **artificial neuron**:
 - has one or more binary (on/off) inputs and one binary output
 - activates its output when more than a certain number (e.g. two in the example below) of its inputs are active



Linear threshold unit (LTU)

- A **linear threshold unit (LTU)** is a type of artificial neuron, where the inputs and output are numbers, instead of binary on/off values
 - Compute a **weighted sum** of inputs
 - Apply a **step function** to the sum to get the output



Common step functions:

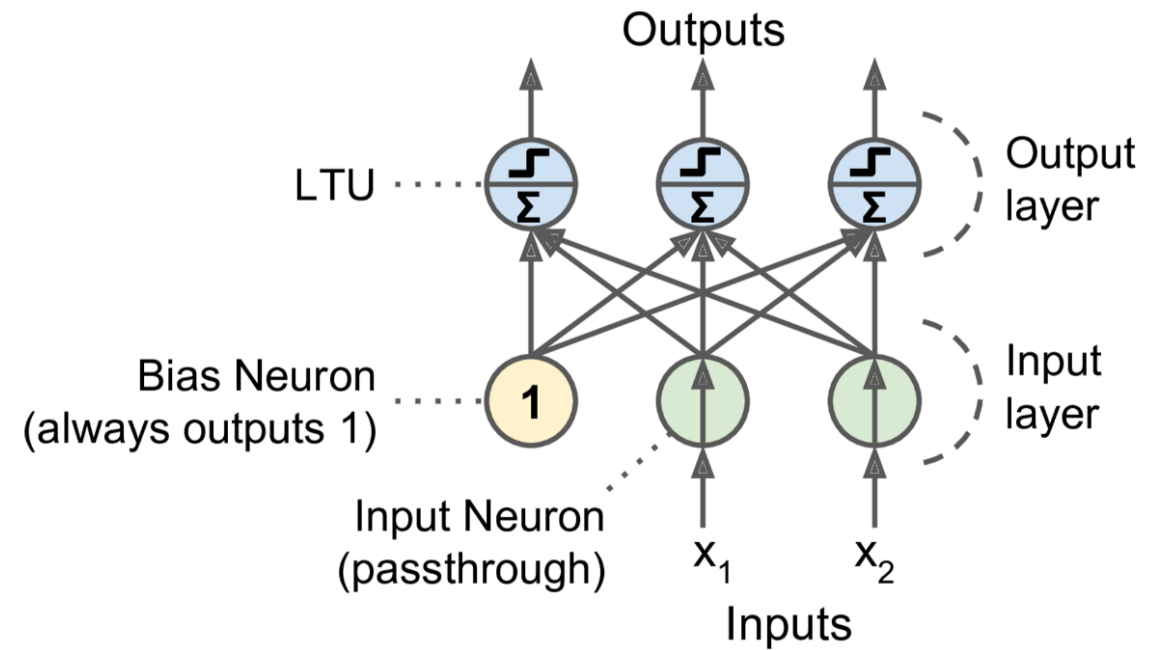
$$\text{heaviside}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

$$\text{sgn}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

Perceptron

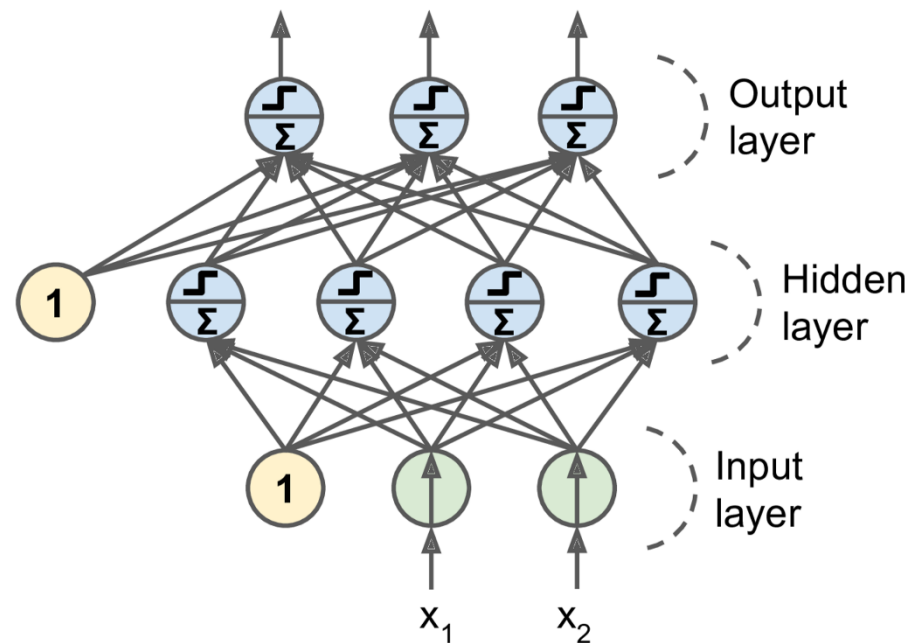
- A **perceptron** (invented by Frank Rosenblatt in 1957) is a simple ANN composed of a single layer of LTUs
 - Each neuron is connected to all the inputs
 - **Input neurons** output whatever input they are fed
 - **Bias neuron**: output 1 all the time
- A Perceptron is trained using **Hebb's rule**:
 - Between two neurons having the same output, increase the connection weight
 - Not to reinforce connections that lead to wrong output

Perceptron diagram:



Multi-Layer Perceptron (MLP)

- Perceptrons are unable to solve some trivial problems (e.g. the XOR classification problem)
- But some limitations of Perceptrons can be overcome by stacking multiple Perceptrons, resulting in the **Multi-Layer Perceptron (MLP)**
- An MLP is composed of
 - One **input layer**
 - One or more **hidden layers** (of LTUs)
 - One final **output layer** (of LTUs)



A Multi-Layer Perceptron



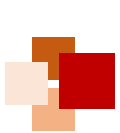


Deep neural network (DNN)



- When an ANN has two or more hidden layers, it is called a **deep neural network (DNN)**
- However, it was difficult to train MLPs for many years, until the introduction of the **backpropagation** algorithm in 1986





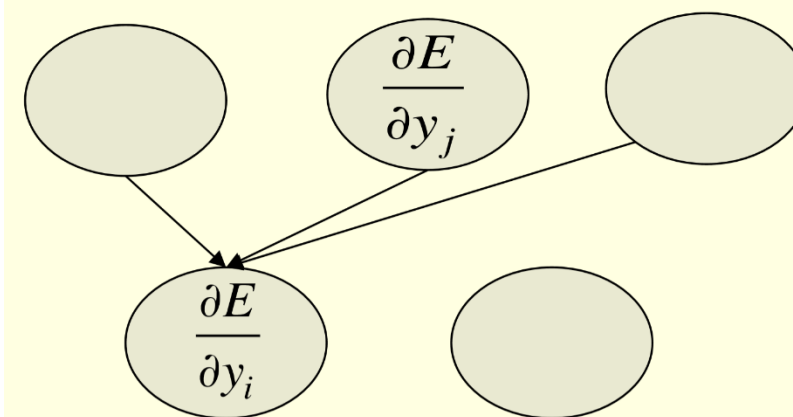
Backpropagation



- For each training instance, the backpropagation algorithm goes through 3 steps:
 - **Forward pass**: feed the training instance to the network, and compute the output of every neuron in each consecutive layer, i.e. to make a prediction
 - **Reverse pass**: measure the error (i.e. the difference between the desired output and the actual output), and go through each layer **in reverse** to measure the error contribution from each neuron in the previous hidden layer, until reaching the input layer
 - **Gradient descent**: slightly tweak the connection weights to reduce the error

$$E = \frac{1}{2} \sum_{j \in \text{output}} (t_j - y_j)^2$$

$$\frac{\partial E}{\partial y_j} = -(t_j - y_j)$$



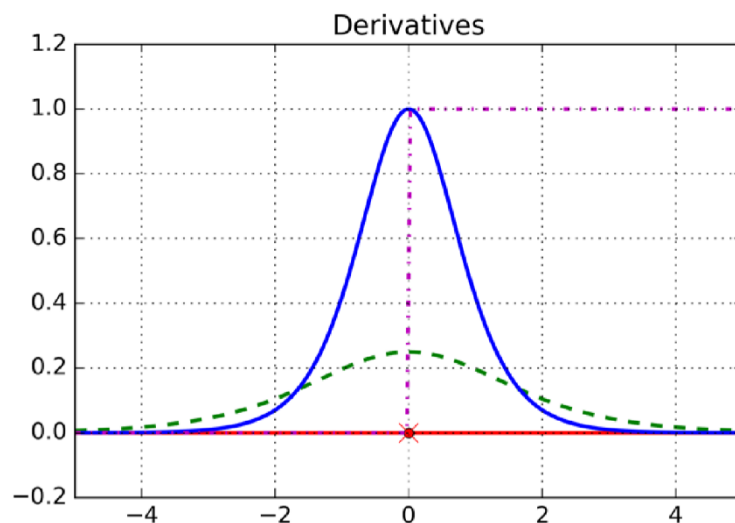
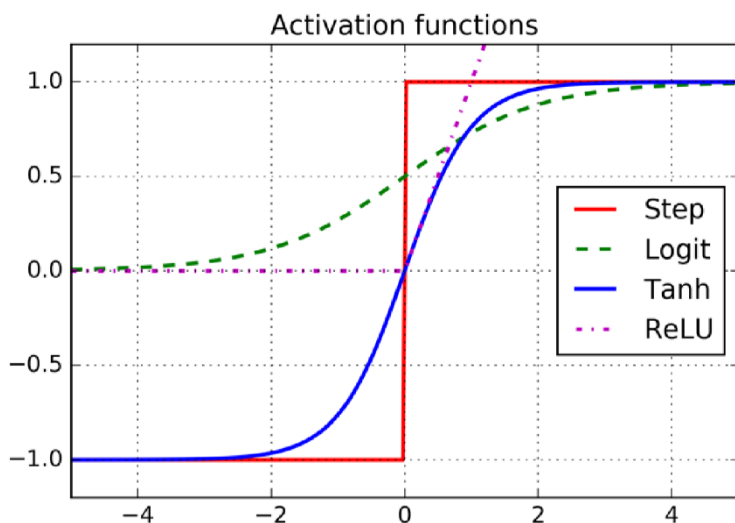
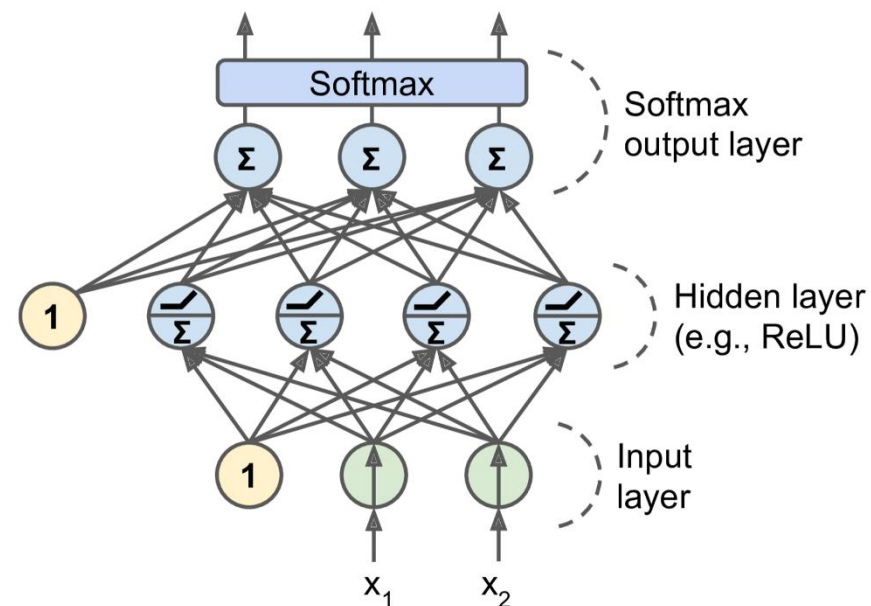
Error derivative in each hidden layer can be computed from the error derivatives in the layer above (i.e. backpropagating dE/dy)



New activation functions

- For the backpropagation algorithm to work properly, the step function is often replaced with other activation functions:

- Logistic functions:** $\sigma(z) = 1/(1 + \exp(-z))$
- Hyperbolic tangent function:** $\tanh(z) = 2\sigma(2z) - 1$
- ReLU function:** $\text{ReLU}(z) = \max(0, z)$

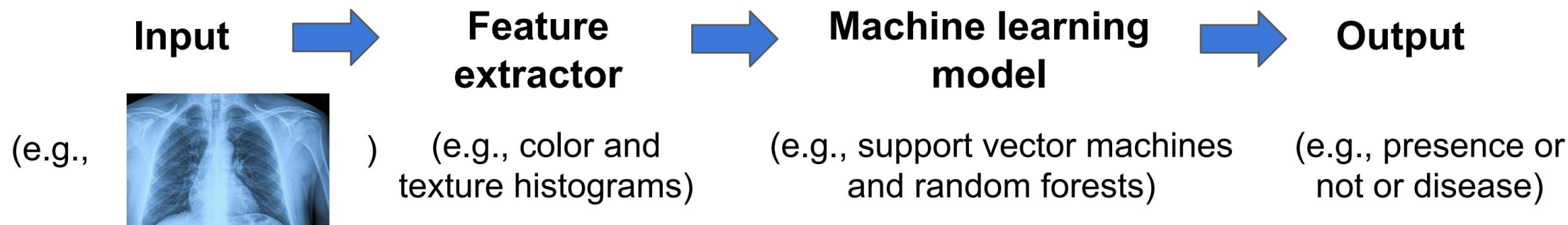




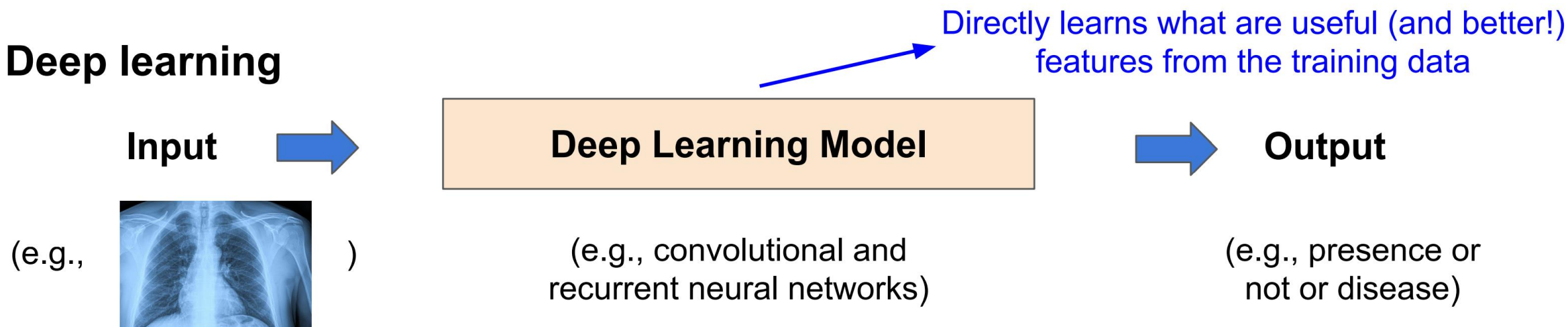
Deep learning vs. traditional machine learning



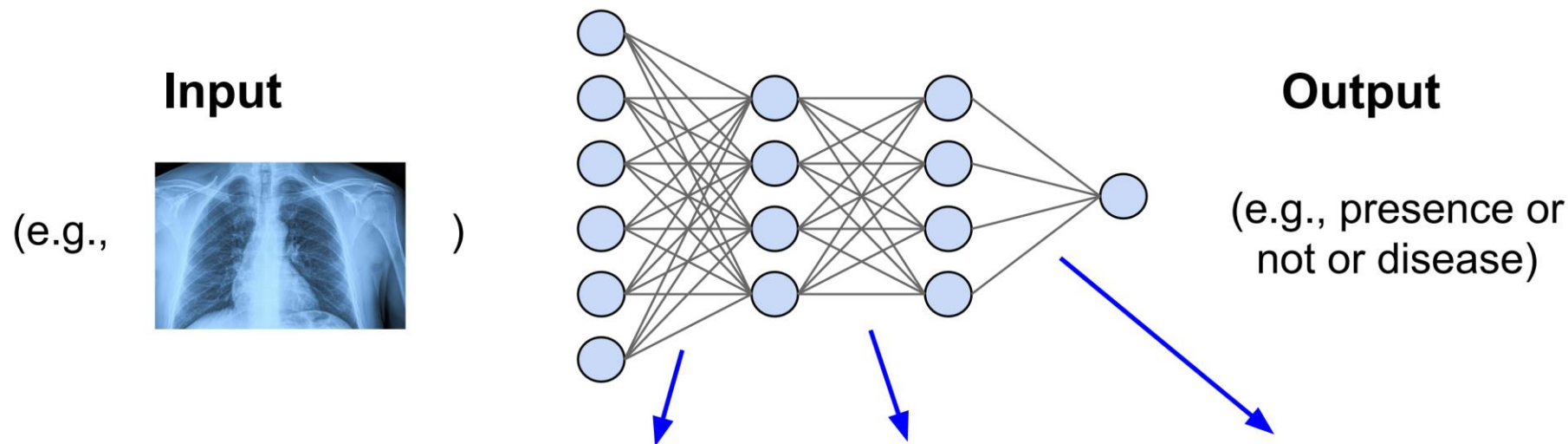
Traditional machine learning



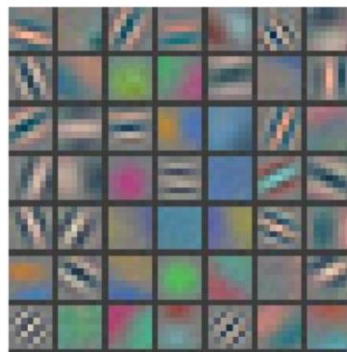
Deep learning



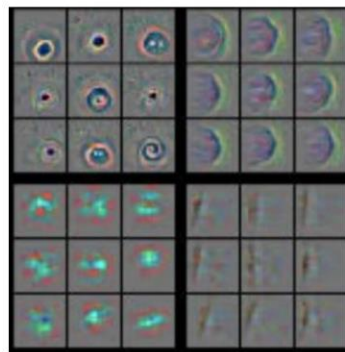
How do DL models perform feature extraction?



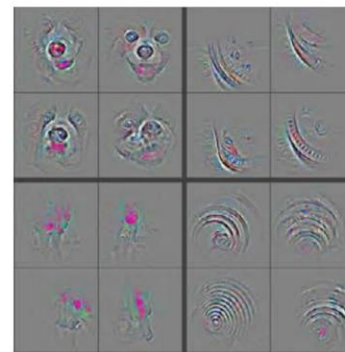
Hierarchical structure of neural networks allows compositional extraction of increasingly complex features



Low-level features



Mid-level features



High-level features

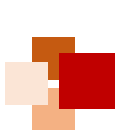
Feature visualizations from Zeiler and Fergus 2013

Slide from Serena Yeung, BIODS220, Stanford



Fine-tuning Neural Network Hyperparameters





Hidden layers



- Real-world data often have **hierarchical structures**, which can be captured by different layers of a deep neural network:
 - **Lower hidden layers** model low-level structures (e.g. line segments of various shapes and orientation)
 - **Intermediate hidden layers** combine the low-level structures to model intermediate-level structures (e.g. squares, circles)
 - **Highest hidden layers** and the **output layer** combine the intermediate structures to model high-level structures (e.g. faces)
- The hierarchical architecture can improve the ability of networks to **generalize** to new datasets, because the low-level structures can be shared and reused





Number of hidden layers

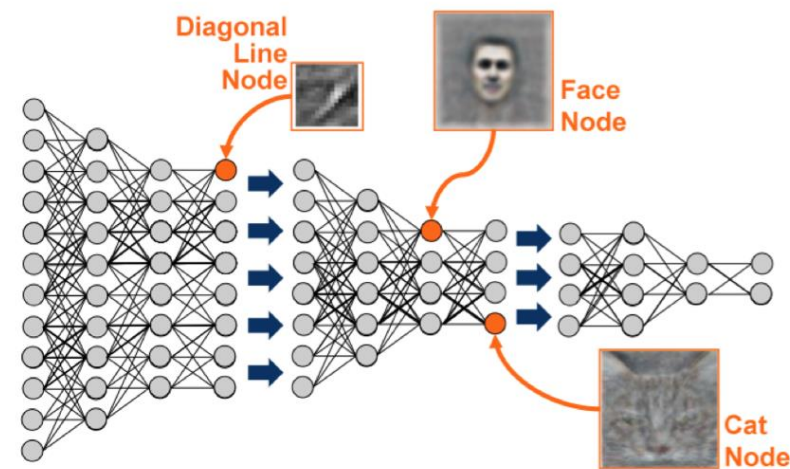


- For many simple problems, a single hidden layer will give reasonable results
- For more complex tasks, however, more hidden layers will be better, because:
 - Deeper networks have higher **parameter efficiency**, i.e. they can model complex functions using exponentially **fewer** neurons than shallow nets, and **speed up** the training
- However, deeper networks **need large amounts of data**, and are prone to **overfitting** due to more parameters and higher model complexity



■ Number of neurons per hidden layer

- The numbers of neurons in the input and output layers are determined by the specific task
- For the hidden layers, a common practice is to form a **funnel**:
 - With fewer and fewer neurons at each layer, because many low-level features can coalesce into fewer high-level features
 - But sometimes, all the hidden layers are set to the same size to reduce the number of parameters
- Strategies to find the number of neurons per layer
 - Try increasing the number of neurons gradually until the network starts overfitting
 - The **“stretch pants”** approach: Pick a model with more layers and neurons than you need, and then use regularization techniques (e.g. early stopping, dropout) to prevent it from overfitting



- In hidden layers, the **ReLU** function (or one of its variants) is often used, because:
 - It is faster to compute than other activation functions
 - It does not saturate for large input values (in contrast to logistic or tanh which saturate at 1)
 - With ReLU, Gradient Descent does not stuck as much on plateaus
- For output layer:
 - Classification: the **softmax** function is generally a good choice
 - Regression: can use no activation function at all





Training Deep Neural Networks





Challenges of training deep neural networks



- To tackle a complex problem, you may need to train a very deep neural network, e.g. with 10 layers or much more, each containing hundreds of neurons
- Challenges of training deep neural networks:
 - The **vanishing gradients** problem (or the related **exploding gradients** problem) that makes lower layers very hard to train
 - There are **not enough training data**, or it is too costly to label the data
 - Training may be extremely **slow**
 - The risk of **overfitting** the training set, when
 - the model has many parameters,
 - there are not enough training instances, or
 - the data are too noisy



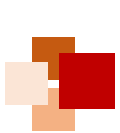


Vanishing/exploding gradients problems



- **Vanishing gradients** problem: In the backpropagation to lower layers, gradients often get smaller and smaller
 - Gradient Descent update leaves the lower-layer weights virtually unchanged, and training cannot converge to a good solution
- **Exploding gradients** problem: The gradients grow bigger and bigger
 - Many layers get large weight updates
 - The algorithm diverges
 - Mostly happens in recurrent neural networks

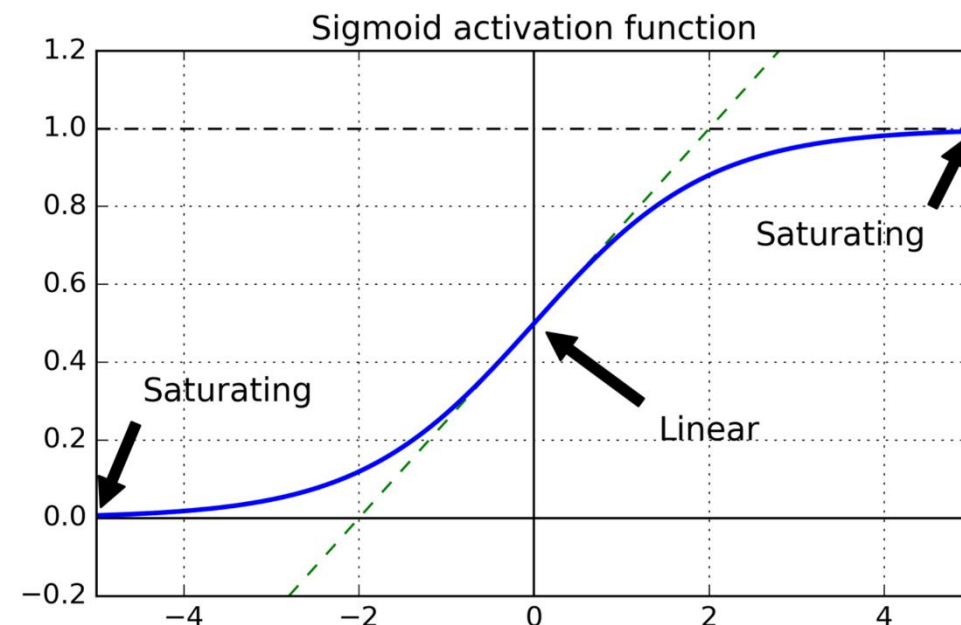




Vanishing/exploding gradients problems



- Reasons found by Xavier Glorot and Yoshua Bengio, around 2010:
 - With the combination of **random initialization with a normal distribution** and **logistic sigmoid activation function**, the variance of outputs of each layer is much bigger than the variance of its inputs
 - The activation function tends to saturate at the top layers



Saturation of logistic activation function





Glorot and He initialization

- **Glorot initialization** (or **Xavier initialization**)

- For each layer, the numbers of input and output neurons are called **fan_{in}** and **fan_{out}** of the layer
- The connection weights of each layer must be initialized randomly as described in the equations on the right side, where

$$\text{fan}_{\text{avg}} = (\text{fan}_{\text{in}} + \text{fan}_{\text{out}})/2$$

- **LeCun initialization**: Replace **fan_{avg}** with **fan_{in}** in the equations of Glorot initialization
- **He initialization**: For RELU and its variants

Glorot initialization with logistic activation function

Normal distribution with mean 0 and variance $\sigma^2 = \frac{1}{\text{fan}_{\text{avg}}}$

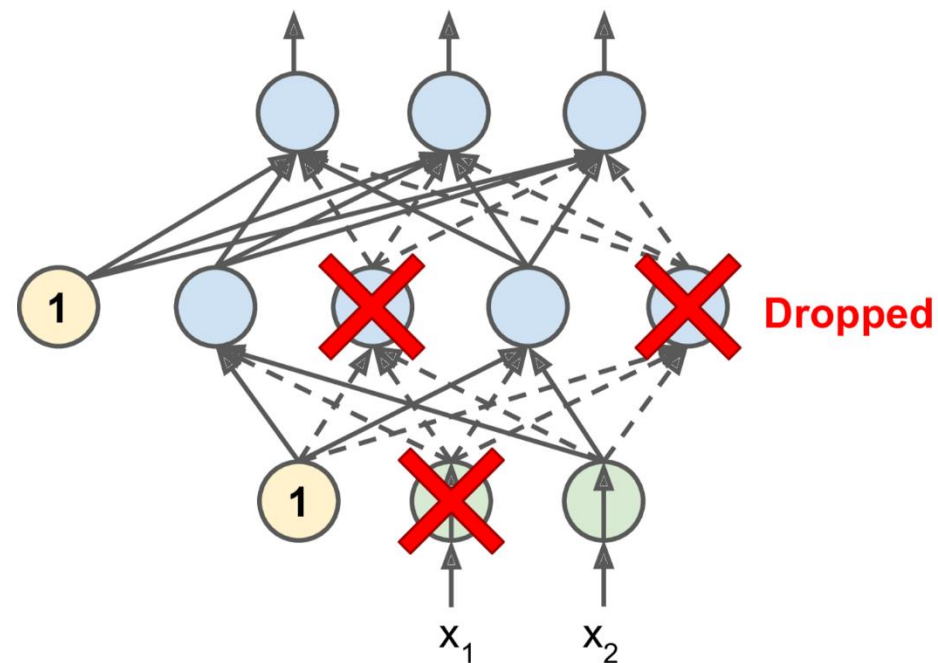
Or a uniform distribution between $-r$ and $+r$, with $r = \sqrt{\frac{3}{\text{fan}_{\text{avg}}}}$

Initialization for different activation functions

Initialization	Activation functions	σ^2 (Normal)
Glorot	None, Tanh, Logistic, Softmax	$1 / \text{fan}_{\text{avg}}$
He	ReLU & variants	$2 / \text{fan}_{\text{in}}$
LeCun	SELU	$1 / \text{fan}_{\text{in}}$

Dropout to avoid overfitting

- The **dropout algorithm**:
 - At every training step, every neuron (except for output neurons) has a probability p of being temporarily “dropped out”
 - The hyperparameter p is called the **dropout rate**, and is typically set to 50%
 - After training, neurons don’t get dropped any more
- Why dropout works?
 - Neurons trained with dropout cannot co-adapt with their neighboring neurons
 - Neurons are forced to pay attention to **all** input neurons, and thus be less sensitive to small changes in the inputs





Convolutional Neural Network (CNN)

PPT courtesy from Prof. Xuming He, SIST, ShanghaiTech University





Biological background: Visual Cortex



- David H. Hubel and Torsten Wiesel performed experiments on the visual system of cats in 1958 and 1959 (Nobel Prize in Physiology or Medicine, 1981)
- They gave insights into the structure of visual cortex:
 - Many neurons in the visual cortex have a small **local receptive field**, i.e. they react only to visual stimuli located in a limited region of the visual field; together they tile the whole visual field
 - Some neurons react only to lines of specific orientations (e.g. horizontal lines)
 - Some neurons have larger receptive fields and react to more complex patterns that are **combinations of lower-level patterns**
 - **Crucial idea:** Higher-level neurons are based on the outputs of neighboring lower-level neurons

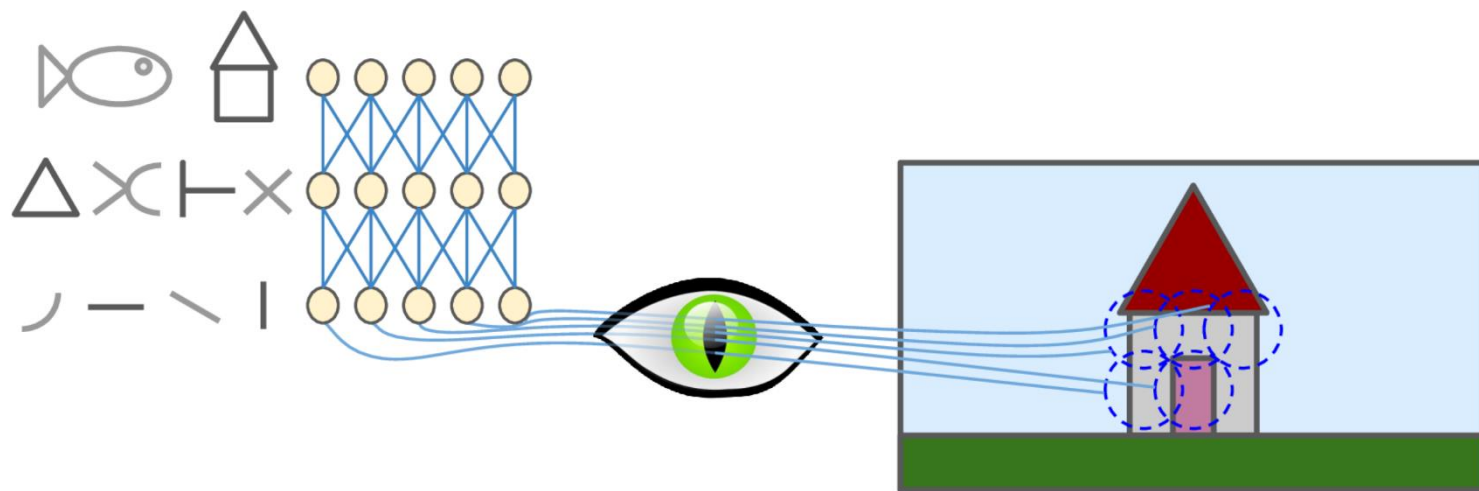
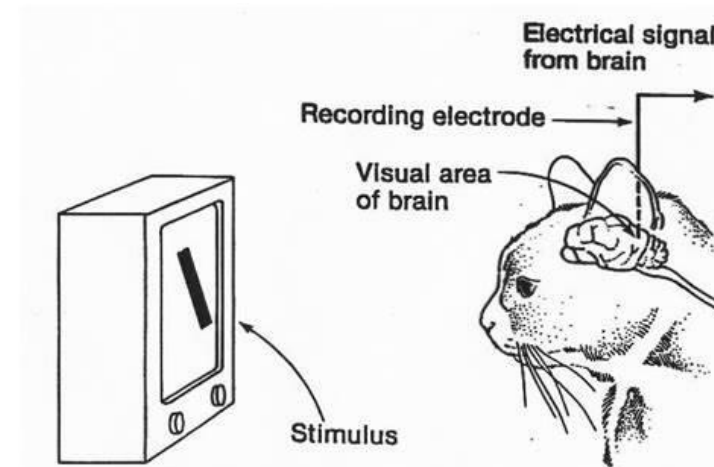


Figure from Fig. 14-1, page 447, Aurélien Géron's book "Hands-on Machine Learning" (Ed. 2), 2019

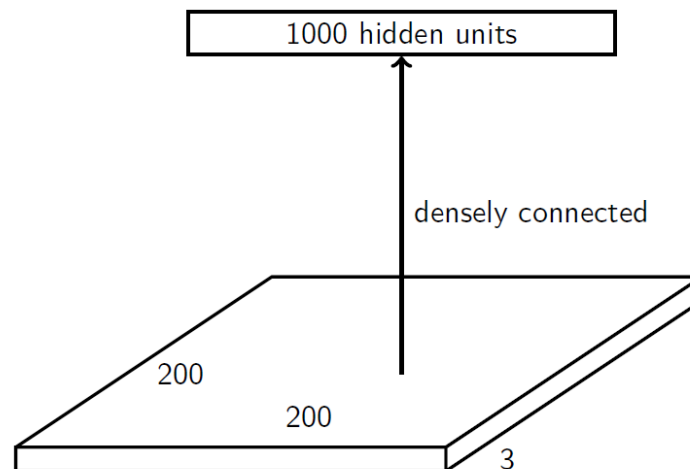




Motivation: Visual recognition



- Suppose we aim to train a network that takes a 200x200 RGB image as input



- What is the problem with having full connections in the first layer?
 - Too many parameters! $200 \times 200 \times 3 \times 1000 = 120$ million
 - What happens if the object in the image shifts a little?

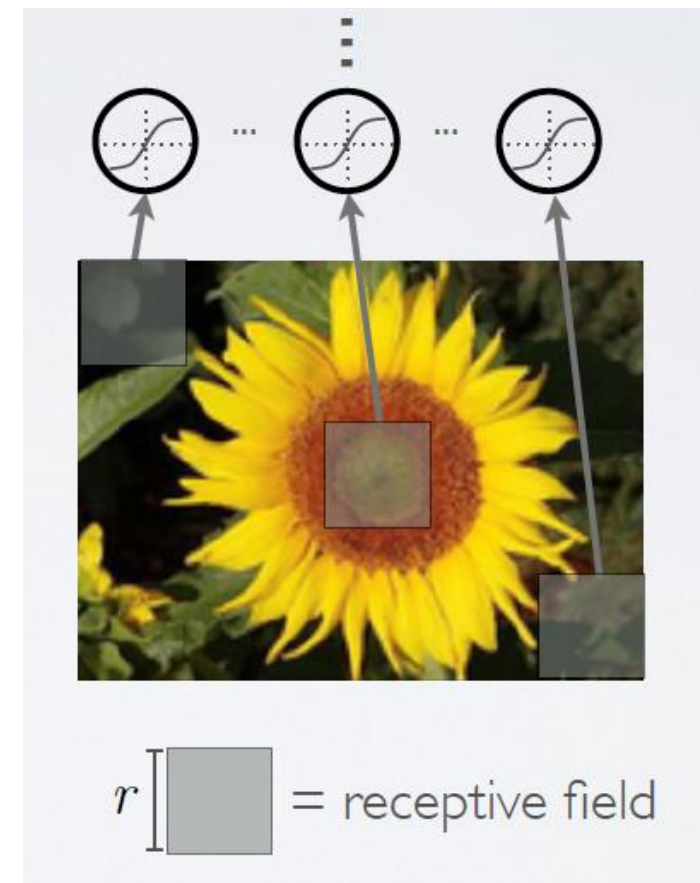


- Visual Recognition: Design a neural network that
 - Can deal with very **high-dimensional inputs**
 - Can exploit the **2D topology** of pixels in images
 - Can build **invariance to certain variations** so that we can expect
 - Translation, small deformations, illumination, etc.
- Convolution networks leverage these ideas
 - Local connectivity
 - Parameter sharing
 - Pooling/subsampling hidden units

Overview of CNNs



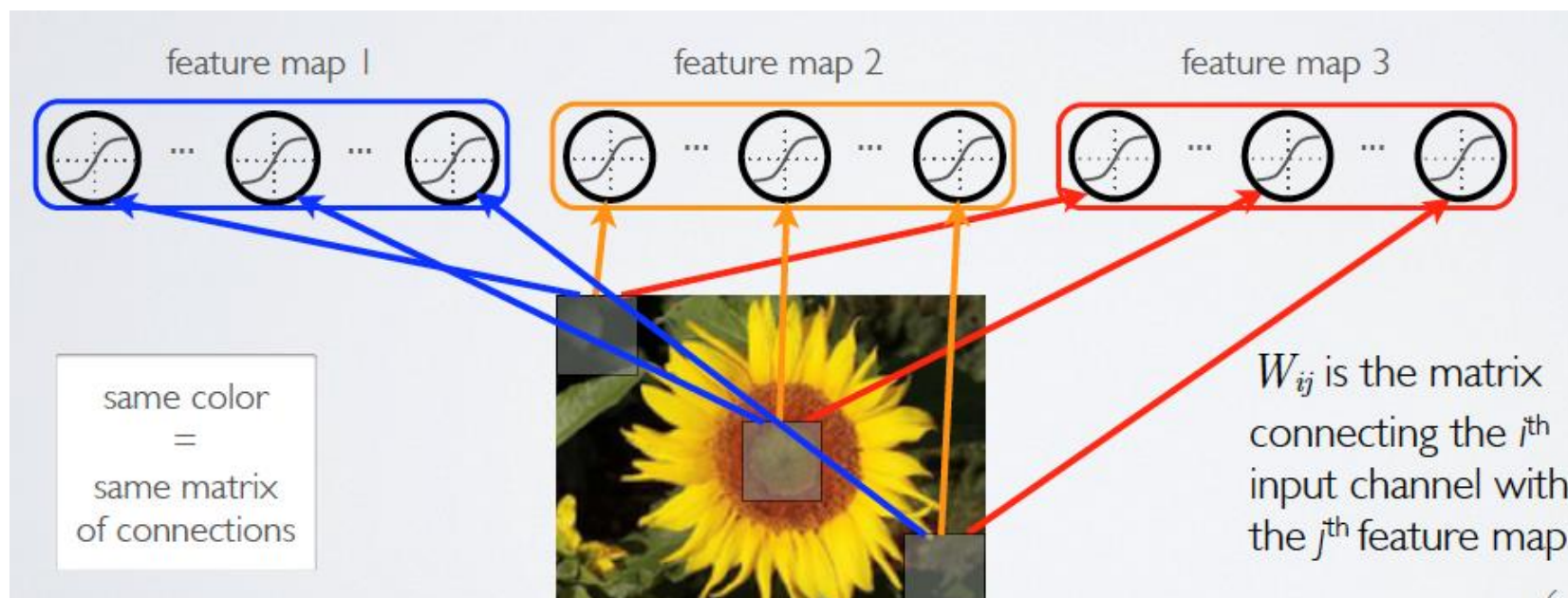
- **Idea 1:** Use a local connectivity of hidden units
 - Each hidden unit is connected only to a sub-region (patch) of the input image
 - Usually it is connected to all channels
 - Each neuron has a **local receptive field**



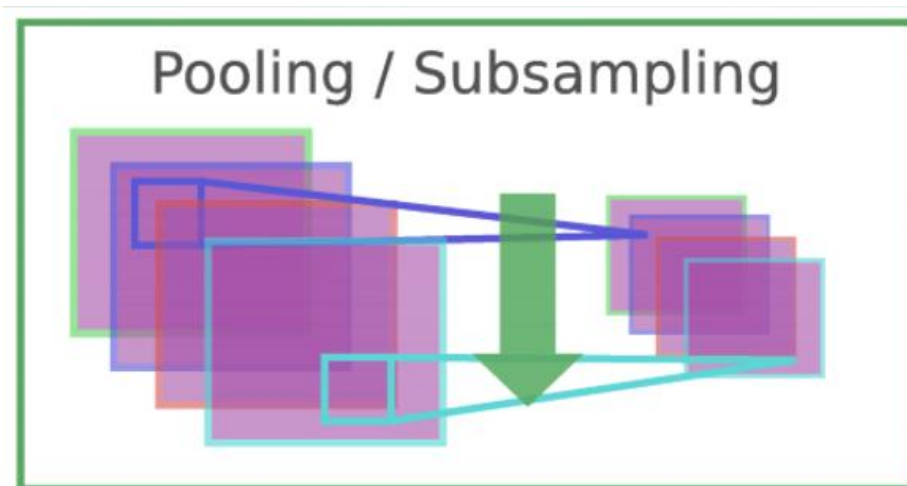
Overview of CNNs



- **Idea 2:** Share weights across certain units
 - Units organized into the same “feature map” share weight parameters
 - Hidden units within a feature map cover different positions in the image



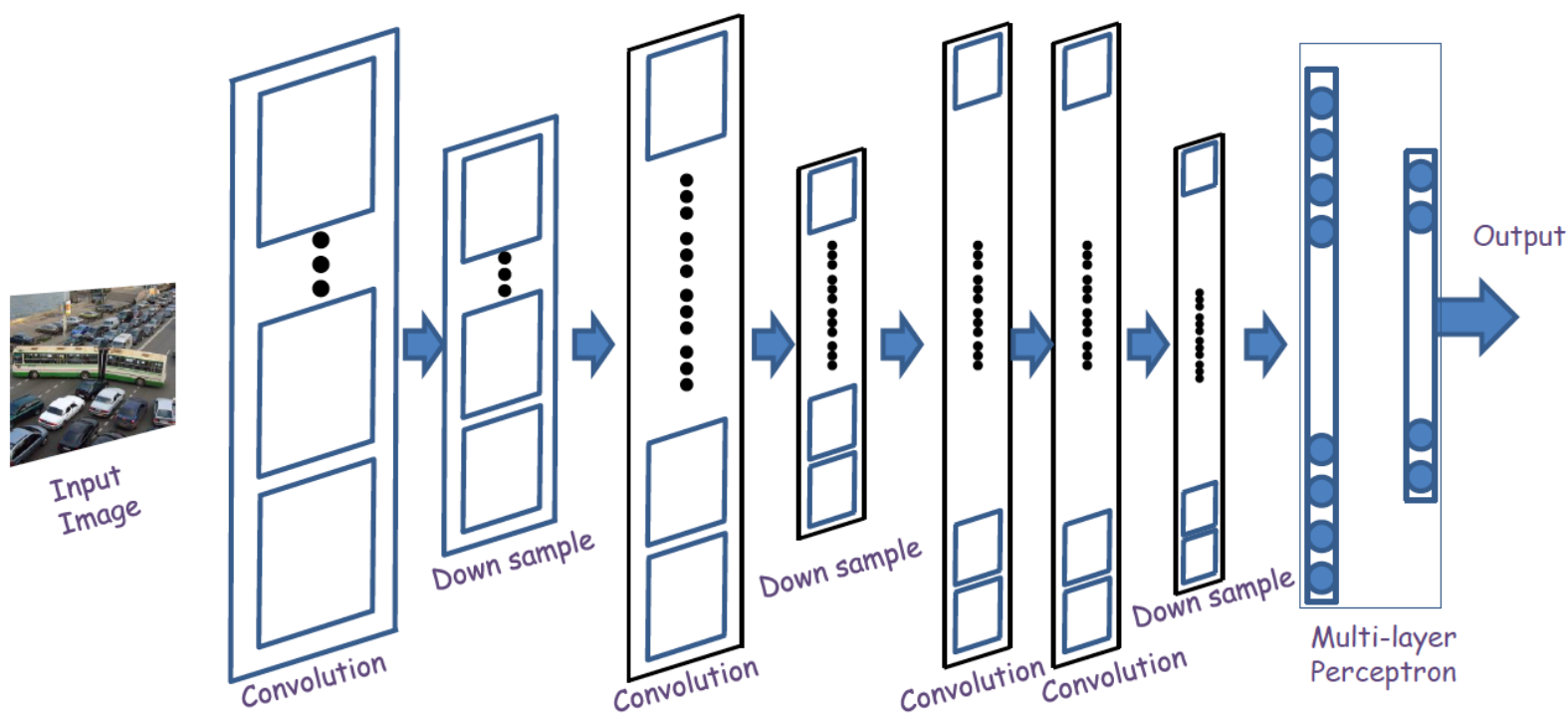
- **Idea 3:** Pool hidden units in the same neighborhood
 - Averaging or discarding location information in a small region
 - Robust toward small deformations in object shapes by ignoring details



Overview of CNNs



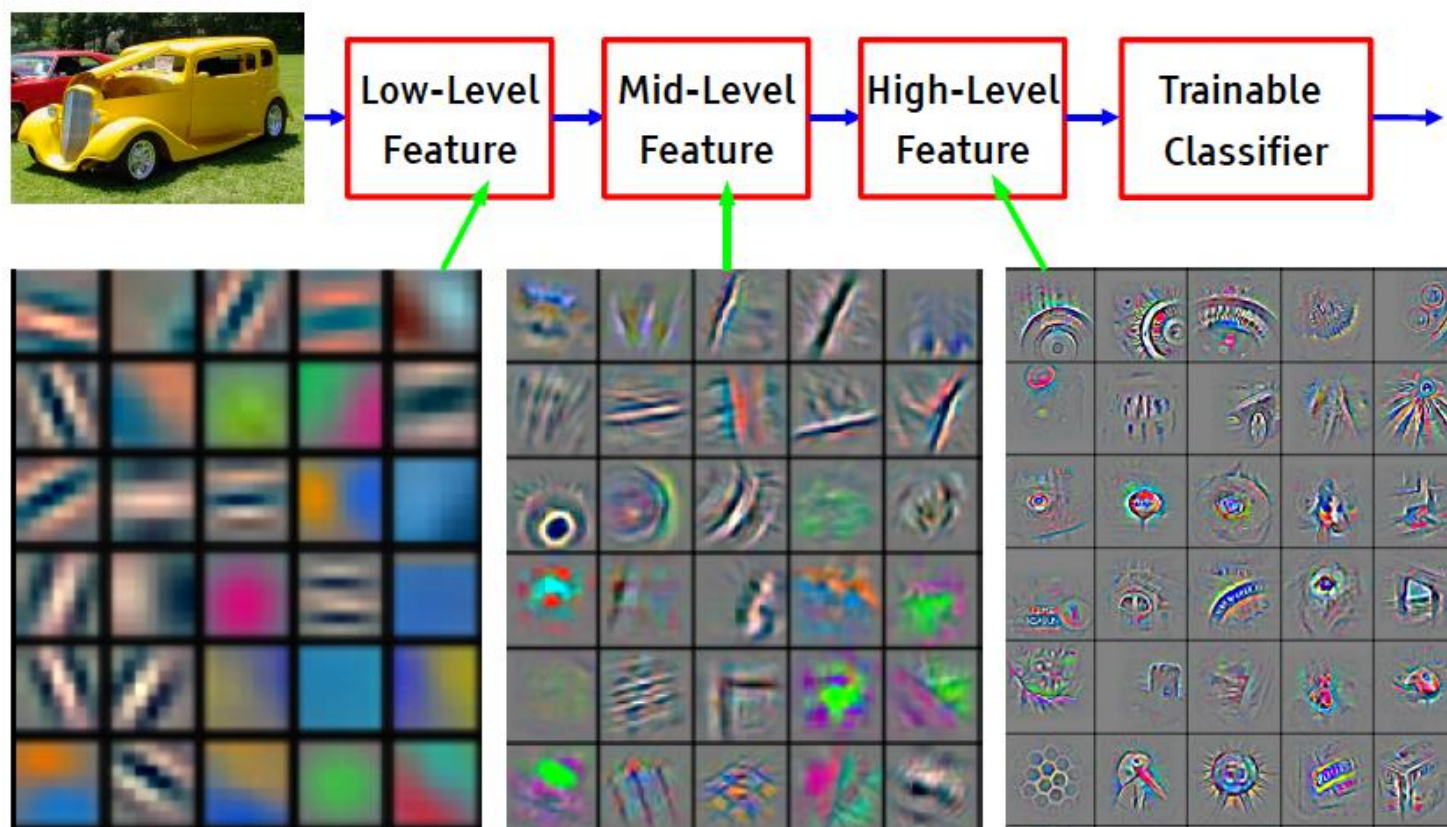
- **Idea 4:** Interleaving feature extraction and pooling operations
 - Extracting abstract, compositional features for representing semantic object classes



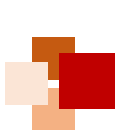
Overview of CNNs



- Artificial visual pathway: from images to semantic concepts
 - Representation learning



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]



2D Convolution



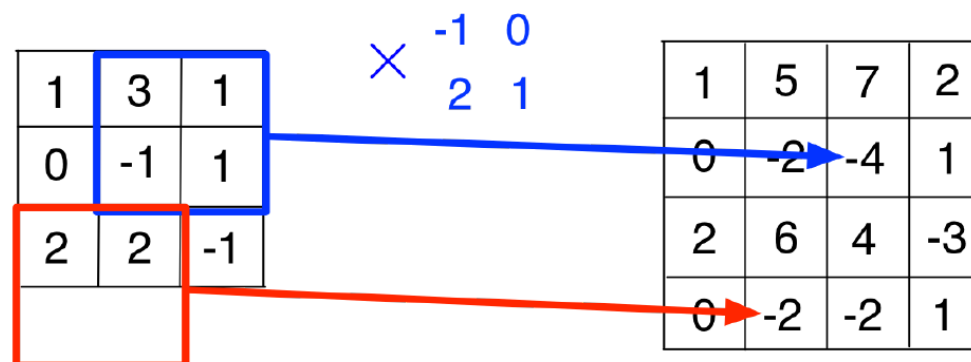
If A and B are two 2-D arrays, then:

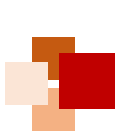
$$(A * B)_{ij} = \sum_s \sum_t A_{st} B_{i-s, j-t}.$$

1	3	1
0	-1	1
2	2	-1

 $*$

1	2
0	-1



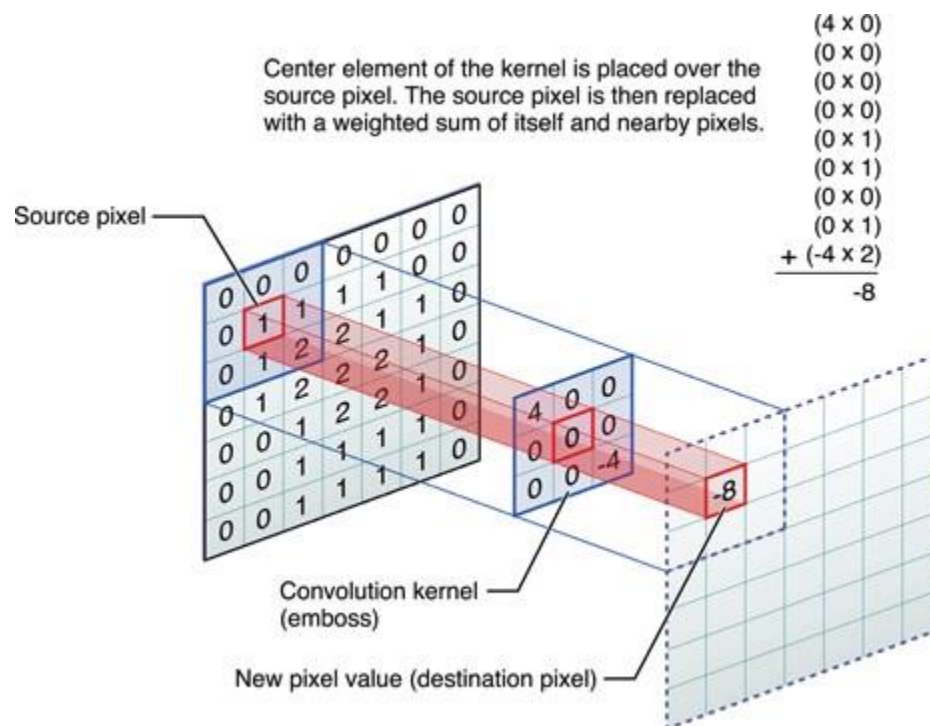


2D Convolution



If A and B are two 2-D arrays, then:

$$(A * B)_{ij} = \sum_s \sum_t A_{st} B_{i-s, j-t}.$$



1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

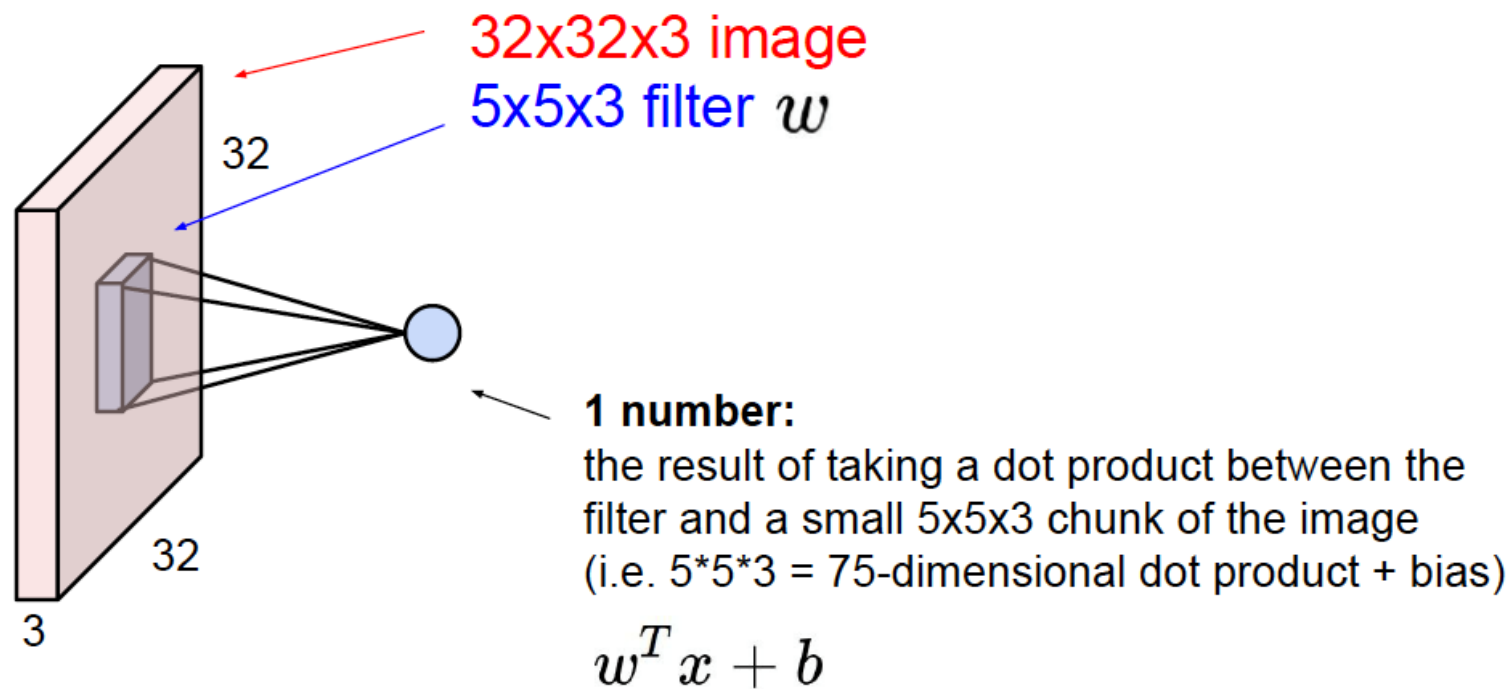
Convolved
Feature

Picture Courtesy: developer.apple.com

Convolution layers



- Formal definition

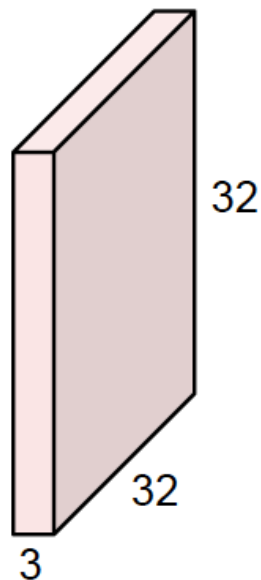


Convolution layers



- Define a neuron corresponding to a $5 \times 5 \times 3$ filter

$32 \times 32 \times 3$ image



$5 \times 5 \times 3$ filter

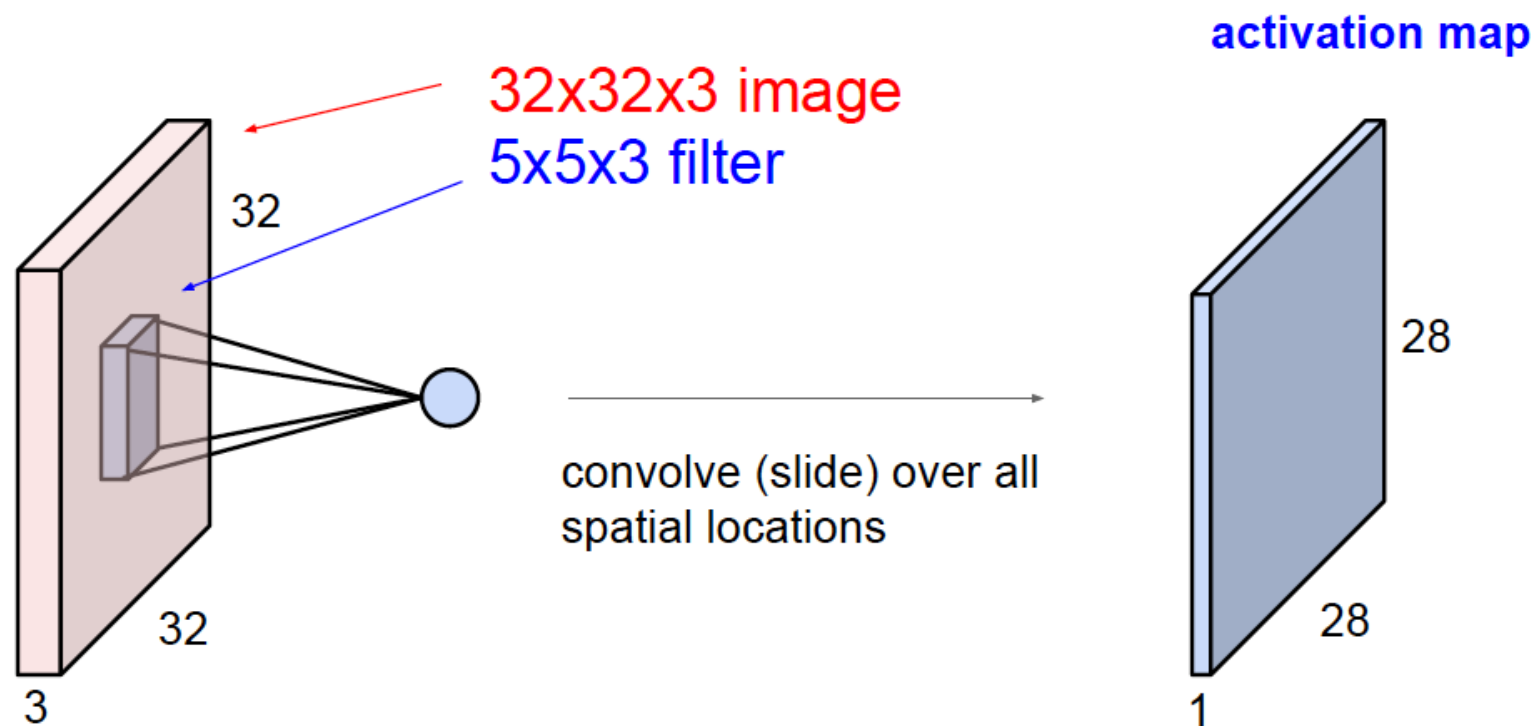


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution layers



- Convolution operation
 - Parameter sharing
 - Spatial information

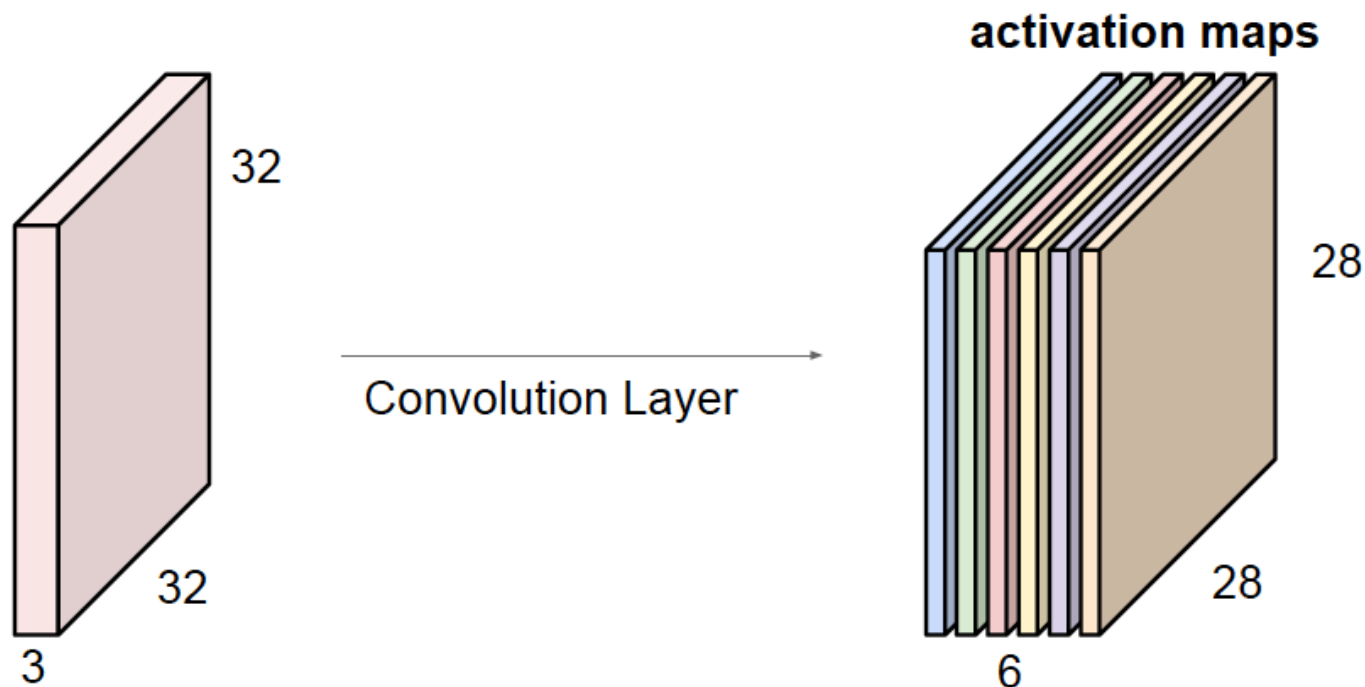


Convolution layers



- Multiple kernels/filters

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



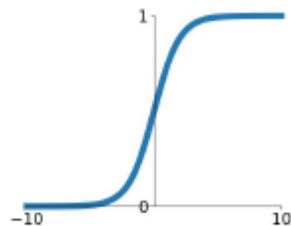
We stack these up to get a “new image” of size 28x28x6!

Zoo of activation functions



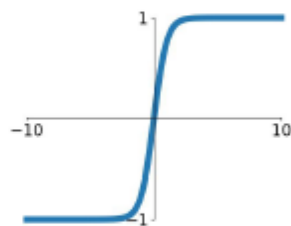
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



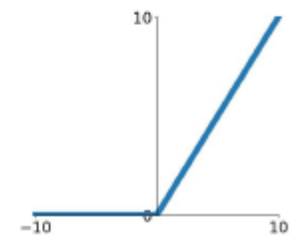
tanh

$$\tanh(x)$$



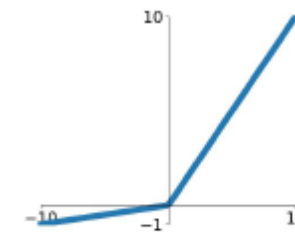
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

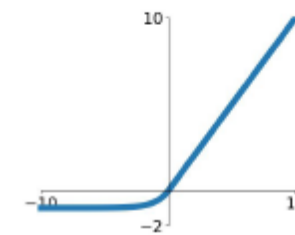


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



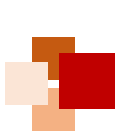


Tips on activation functions



- For internal layers in CNNs
 - Use ReLU. Be careful with your learning rates
 - Try out Leaky ReLU/Maxout/ELU
 - Try out tanh but do not expect much
 - Do not use sigmoid
- For output layers
 - Task dependent
 - Related to your loss function

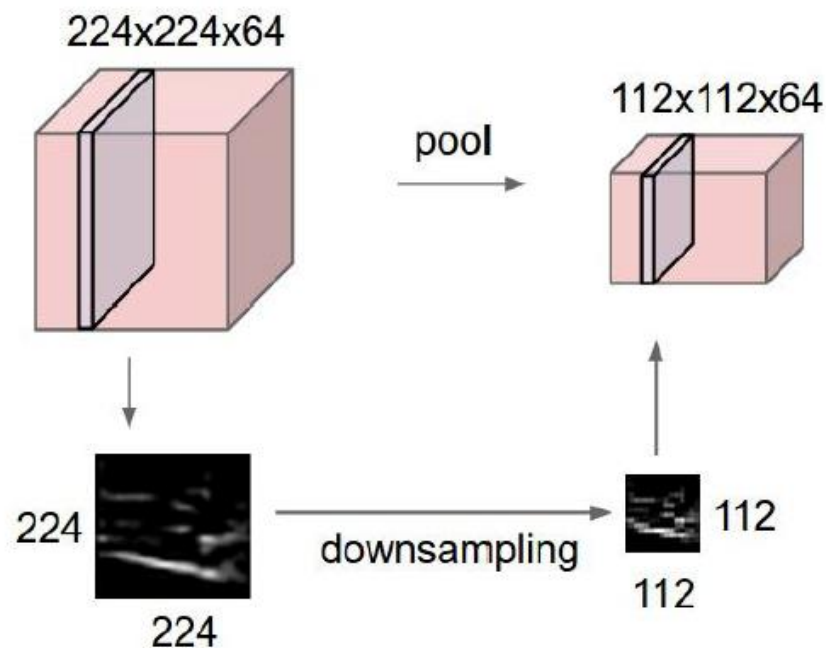




Pooling layers



- Reducing the spatial sizes of the feature maps
 - Smaller representations
 - On each activation map independently
 - Low resolution means fewer details

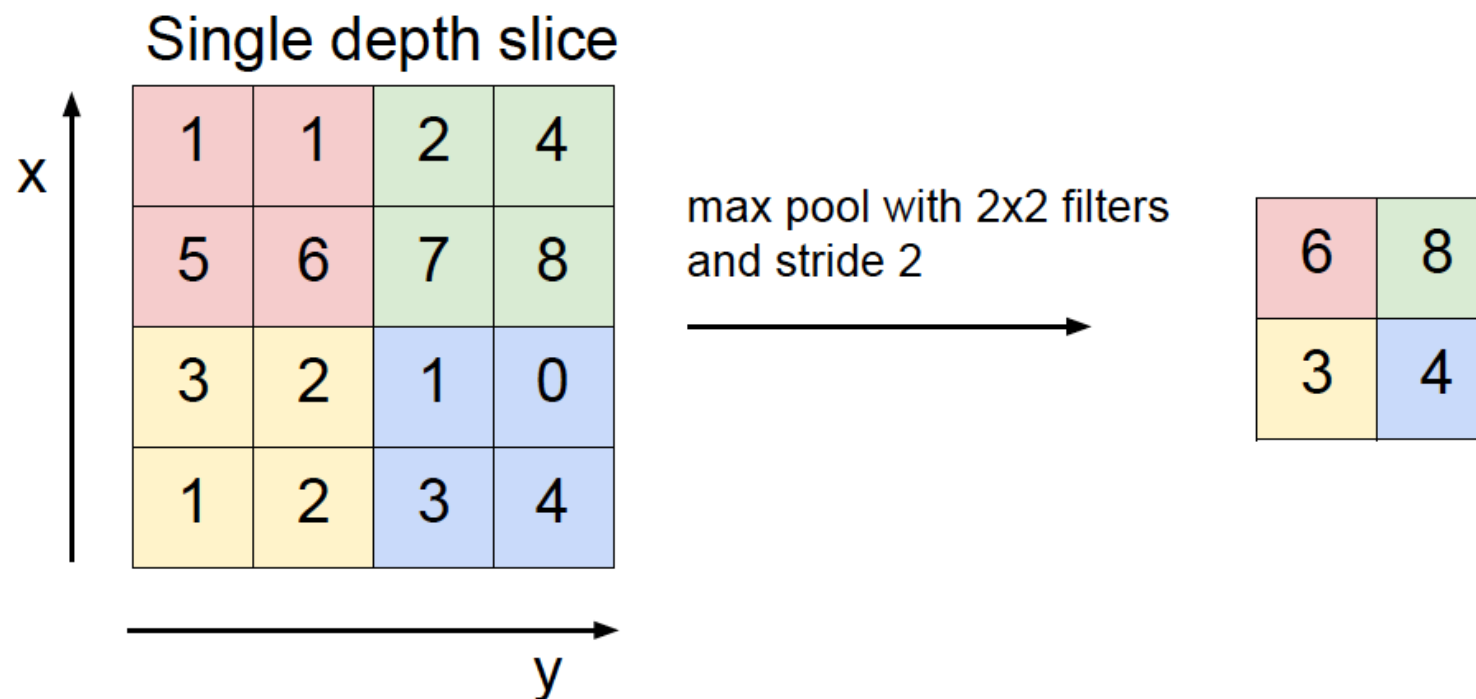




Pooling layers



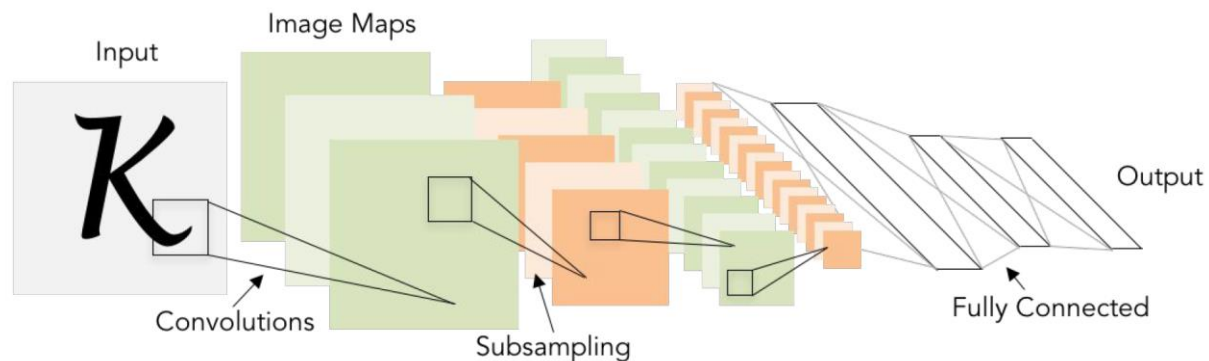
- Example: max pooling



Example CNNs

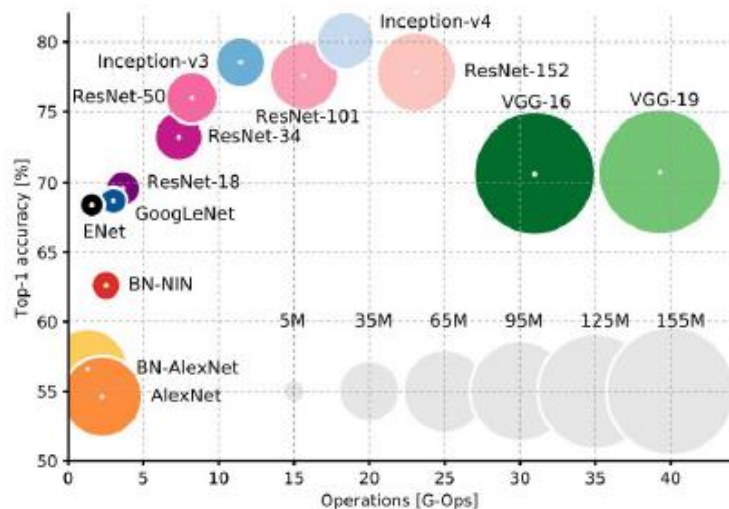
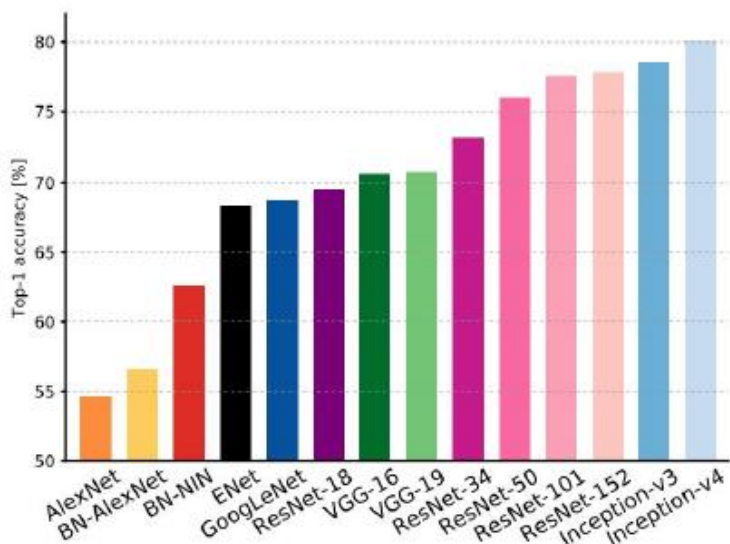
- LeNet-5 (Handwritten digit recognition)
- Sequential structure: AlexNet / VGGNet
- Parallel branches: GoogLeNet
- Residual structure: ResNet / DenseNet

[LeCun et al., 1998]



Conv filters were 5x5, applied at stride 1
Subsampling (Pooling) layers were 2x2 applied at stride 2
i.e. architecture is [CONV-POOL-CONV-POOL-FC-FC]

Increase of model complexity



An Analysis of Deep Neural Network Models for Practical Applications, 2017.



References



- Aurélien Géron' s book “Hands-On Machine Learning with SciKit-Learn, Keras & TensorFlow” , Ed. 2 (2019), Chapters 10, 11, 14
- Book “Dive into Deep Learning” , Chapter 14 “Computer Vision” :
 - http://d2l.ai/chapter_computer-vision/index.html
- Chapter 9 of Ian Goodfellow et al. “Deep Learning” , MIT Press, 2017.
- Yann LeCun et al. “Gradient-based learning applied to document recognition” , Proceedings of the IEEE 86, no. 11 (1998): 2278-2324.





Recap



- In this lecture we have learned
 - Analogy between biological neurons and artificial neurons
 - How to train a neural network
 - The vanishing / exploding gradients problems and techniques to address them
 - How to avoid overfitting by regularization techniques (e.g. dropout)
 - What is CNN, and how does it work?
 - Typical CNN structures

